Supplementary Material: Reproducibility and Proof-Audit Notes

Saveliy Baturin

This supplement accompanies the manuscript "From Approximation Rates to Loss-Landscape Barrier Decay in Shallow ReLU Networks."

Status of the computational diagnostic

The exploratory experiments use Dynamic String Sampling (DSS), a recursive construction of a piecewise-linear path introduced by Freeman and Bruna in *Topology and Geometry of Half-Rectified Network Optimization* (ICLR 2017, `https://openreview.net/forum?id=BkOFWVcgx`). Their greedy algorithm inserts an interpolated model at a high point of a segment, trains that model below a prescribed level, and recursively treats the two new segments. Our journal implementation preserves this search principle but is not an unmodified copy: it uses adaptive grids, a global-Lipschitz mesh correction, explicit terminal statuses, and a final re-evaluation of every returned segment. A reproducibility audit found that the previous local implementation reported the minimum among unresolved terminal-segment exceedances when recursion stopped. The objective of a returned path must instead be its maximum over all final segments. The shared implementation in `dss.py` and the theorem-aligned implementation in `theory_experiments/paths.py` now report that maximum.

The old notebook CSV files were produced by the earlier aggregation and cannot be corrected without the intermediate trained paths. They are retained as historical development artifacts but are not used as evidence in the revised manuscript. The reported tables and figures instead come from the frozen theory-aligned run completed on 12 August 2026 and stored in the three `reviewer_main_gpu_n*` result directories.

Rerun protocol

The two historical notebook pipelines are:

1. `0_energy_gap_experiment.ipynb` for Two Moons regression;
2. `1_energy_gap_cancer_experiment.ipynb` for breast-cancer classification.

The shared DSS implementation is in `dss.py`. The previous configuration, which can be used as a baseline for the corrected rerun, was:

1. widths $[20, 200]$;
2. 1000 independently trained models per width and 200 sampled pairs;
3. output-layer ℓ_1 coefficient 0.001;
4. maximum pool-training and DSS midpoint-training budgets of 150 epochs;
5. DSS recursion depth 8 and 200 evaluation points per segment;
6. pair-specific threshold equal to the larger endpoint objective;

7. first-layer normalization enabled;

8. 2000 bootstrap replicates and 2000 permutations.

For a journal rerun, at least four widths should be used. Each output row should contain the returned-path maximum, whether the recursion limit was reached, the number of terminal segments, and the endpoint objective values. A run that reaches the recursion limit remains a computed path with a measurable barrier, but it is not a certificate that no lower path exists.

Canonical theory-aligned pipeline

The journal experiment is implemented in `theory_experiments/` and configured by versioned JSON files. The bias-free network, objective evaluator, deterministic cube-cell compression, proof-inspired path, and DSS certificate are implemented directly with NumPy. The frozen main run uses the PyTorch backend only to batch independent projected-proximal-Adam trajectories on a CUDA device. Every stored result is converted to double precision, projected once more onto the unit ball to remove single-precision boundary drift, and re-evaluated by the same NumPy objective used for all path certificates. Thus GPU batching affects execution but not the reported evaluator or parameter domain.

The execution order is:

1. run `python -m unittest discover -s tests -v`;

2. run `configs/reviewer_smoke.json` as a software check;

3. run `configs/reviewer_pilot.json` to calibrate numerical resolution and budget;

4. freeze and run `configs/reviewer_main_gpu_n1.json`, `n2.json`, and `n4.json`;

5. combine the three result directories with the module `analyze_main_results.py`.

The main design uses dimensions $1, 2, 4$, widths $8, 16, 32, 64, 128$, ten independent pool replicates, eight models per pool, and three disjoint pairs at each of the fixed and moving levels. Pairing is disjoint within a level group, not necessarily across the two levels. The fixed level is common across widths. Moving levels are selected from a prespecified within-replicate order statistic plus a deterministic width-decaying slack. The implementation records the expected and achieved number of pair rows; an incomplete design is reported rather than silently treated as balanced.

For every trained endpoint, the code records the active support, output ℓ_1 norm, maximum first-layer row norm, objective components, deterministic cluster diameter, observed compression increment, and the explicit ambient-cube certificate stated in the manuscript. It aborts if a row leaves the unit ball or if a compression record violates its analytic bound. For selected pairs, exact endpoint states and both DSS and proof-inspired path nodes are stored in a compressed array archive.

The statistical unit is a complete independent pool replicate. Pair-level rows are not permuted or bootstrapped as if independent. Descriptive slopes are block-bootstrapped over replicates and suppressed when exact zero gaps make a log–log fit unidentified. The one-dimensional experiment is a saturation control because the bias-free one-dimensional dictionary reduces to two ReLU rays.

Frozen main-run parameters and outputs

The main run uses seed 20260812; dimensions $1, 2, 4$; widths $8, 16, 32, 64, 128$; ten pool replicates; eight models per pool; and three disjoint pairs per level. The empirical distribution has

384 points, teacher width 512, coefficient decay 1.35, and target scale 0.8. The objective is Huber loss with parameter 0.25 plus output coefficient ℓ_1 weight 0.002. Pool training uses 1800 maximum epochs, learning rate 0.015, patience 250, and CUDA float32 optimization. DSS uses depth seven, a final 513-point segment grid, tolerance 10^{-5}, and midpoint training with 1000 maximum epochs and patience 150. The constructive reference widths are $2, 4, 8, 16, 32$, with eight optimizer starts per width.

The loss-robustness rerun uses the same seed, dimensions, widths, replicate and pair counts, architecture, ridge teacher, optimizer budgets, DSS budgets, reference widths, and regularization. Its configurations are `reviewer_cross_entropy_gpu_n1.json`, `n2.json`, and `n4.json`. The only intended changes are deterministic labels $y = \mathbf{1}\{z_{\text{teacher}} \geq 0\}$ and binary cross-entropy with logits, $\log(1+e^u) - yu$. The code records the realized class fraction. Cross-loss plots use the returned gap divided by the selected level because the raw Huber and cross-entropy objectives have different scales; the statistic remains the worst value within an independent pool replicate.

The execution environment was Windows 11, Python 3.12.7, NumPy 2.4.1, PyTorch 2.8.0 with CUDA 12.9, and an NVIDIA GeForce RTX 5070 Ti. For each loss, the three dimension-split runs produced 1200 trained-model rows, 1200 compression rows, and 900 pair rows. Each dimension has exactly 300 pair rows and is marked `complete_pair_design=true`. The aggregate process wall-clock times were 3273.16 seconds for Huber and 7041.79 seconds for binary cross-entropy.

Before the cross-loss figure or table is written, `analyze_loss_robustness.py` requires all 2400 model rows, all 1800 pair rows, and all 2400 compression rows; verifies three disjoint pairs in every replicate–width–level group; checks the unit-ball constraint and every analytic compression inequality; and rejects any selected endpoint above its recorded level by more than the numerical tolerance. The figure separates fixed and moving levels and aggregates only after taking the worst of the three pairs inside each independent replicate.

Each result directory contains the following files:

- `model_records.csv`, `model_summary.csv`, and `compression_records.csv`;
- `pair_records.csv`, `pair_summary.csv`, and `rate_estimates.csv`;
- `metadata.json` and `states_and_paths.npz`.

The array archive stores every trained endpoint and one DSS and constructive path per replicate-width-level group. The analysis script creates `analysis_report.json`, `numerical_summary.csv`, the LaTeX table, and the two submission figures directly from these records.

Interpretation of path certificates

For each direct or DSS segment, let M_G be the maximum on an equally spaced grid of G points and let K be the analytic Lipschitz constant of the objective along that segment. The recorded upper value is

$$M_G + \frac{K}{2(G-1)}.$$

The maximum of these segmentwise values is an upper bound for the returned piecewise-linear path. A positive `unresolved_segments` count means that DSS reached a depth or training limit before certifying the requested tolerance; it does not invalidate the returned-path upper value, and it does not prove that no lower path exists.

The constructive path has no grid error: convexity controls output interpolation at fixed first-layer rows, while relocating zero-output rows and transferring coefficients between duplicate atoms preserve the predictor. Its recorded node maximum therefore bounds the complete

continuous path. Because the common reference network is the best among finitely many optimizer starts rather than a certified global minimizer, this is a valid path upper bound but not a numerical evaluation of the infimum $e(l)$.

The automatically generated finite-range slope file is retained for auditability. Those slopes are not used in the manuscript's numerical claims: the large transition occurs mainly between widths 8 and 16, while the corrected DSS values subsequently plateau near the 10^{-5} tolerance and many constructive gaps are exactly zero. Fitting a power law to that regime would conflate geometric decay, optimizer behavior, and numerical resolution.

Dense-representation stress test

The proximal optimizer makes the trained endpoints sparse enough that four-coordinate compression is a no-op in the wide regime. The separate dense-endpoint analysis module therefore reuses the selected fixed-level Huber endpoints at widths $16, 32, 64, 128$ and controls the representation density directly. First, every active row is sphericalized using positive homogeneity in the penalty-decreasing direction. Its coefficient is then divided among duplicate rows until all m coordinates are active; this preserves the predictor and output ℓ_1 norm exactly. In dimensions two and four, deterministic tangent directions split those duplicates geometrically. The largest amplitude in a frozen 33-point grid from 0 to 0.25 that remains below the fixed level plus the numerical tolerance 10^{-5} is retained. No tangent move is used for the one-dimensional two-ray control. The executable module is `theory_experiments/analyze_dense_endpoint_stress.py`.

For each dense state, the algorithm sets the target support to $m - 4$. If k coefficients must be removed, it forms the $k + 1$ nearest-neighbour candidate around every active row, selects the candidate with smallest Euclidean diameter, and merges all signed coefficients into one representative. This selection rule is a finite stress diagnostic, not the worst-case sphere-cover construction in the theorem. Its post-hoc certificate is nevertheless exact:

$$F_{\text{merged}} - F_{\text{dense}} \leq LD_X \|\theta\|_1 \operatorname{diam}(Q).$$

The test contains 720 records: six fixed-level selected endpoints in every one of $10 \times 4 \times 3$ replicate–width–dimension groups. All coordinates are active before merging, at least four coefficients are removed in every record, and all 720 diameter certificates hold. The largest positive increment is 0.0136675654 and the largest positive-increment/certificate ratio is 0.0395906. The figure and records are generated in `results/dense_endpoint_stress/`. These observations validate the active merge mechanism under controlled density; they are not used to fit or verify an asymptotic exponent.

Dataset preparation

1. Two Moons: `make_moons` with 1000 samples, noise 0.2, and random state 42, followed by an 80/20 split and standardization on the training portion. Mean squared error is outside the global-Lipschitz hypothesis of the theorem.

2. Breast cancer: `load_breast_cancer()`, followed by a stratified 80/20 split and standardization on the training portion. Binary cross-entropy with logits matches the loss-side assumptions more closely.

Statistical reporting

The notebooks use a one-sided Mann–Whitney comparison as a rank/distributional comparison, not as a test of means. Monte Carlo permutation p-values use the add-one correction

$$\widehat{p} = \frac{b+1}{B+1},$$

where b is the number of permuted statistics at least as extreme as the observation and B is the number of permutations. Therefore, with $B = 2000$, the smallest reportable value is exactly $1/2001$, never zero and never smaller than $1/2001$.

Historical legacy-pipeline alignment note

The legacy notebook code includes biases, whereas the theorem is stated for the bias-free model. It also enforces row normalization after optimizer steps rather than Euclidean-ball projection. Numerical evidence from that legacy pipeline should therefore be described as a qualitative diagnostic rather than as a direct instantiation of the theorem.

This warning applies to `dss.py` and the historical notebooks. The canonical pipeline in the `theory_experiments` directory removes both mismatches. The manuscript's reported binary-cross-entropy robustness check uses the synthetic ridge-teacher design, not the historical breast-cancer notebook.